

Rapport de projet

A-CorePilot & Clean Core ABAP

Analyse du patrimoine ABAP et refactorisation du report ALV ZPROG_ALV_YMEProjet : extension VS Code A-CorePilot

Exercice : migration progressive Clean Core d'un report ALV classique

Copie de travail : ZPROG_ALV_YME_CC

Date : 11 septembre 2026

Contexte : travail d'exploration encadré avec validation dans SAP Logon

1. Résumé exécutif

Le travail a porté sur deux axes. Le premier consistait à rendre A-CorePilot capable d'inventorier les objets Z/Y, d'exécuter ATC et de construire un graphe dont les compteurs sont traçables. Le second consistait à appliquer une démarche Clean Core à un report ALV existant sans modifier directement l'original.

- Distinction entre un résultat where-used vide et une analyse échouée ou non supportée.
- Provider ABAP autour de RS_EU_CROSSREF pour obtenir des callers identifiables.
- Métriques du graphe calculées uniquement depuis les relations SAP_CROSSREF.
- Résultats regex, ATC, ADT générique et LLM conservés comme candidats.
- Classification R5 déterministe, explicable et sans LLM.
- Copie, stabilisation et refactorisation du report ALV.
- Recherche d'APIs Released et confinement des accès fiscaux legacy derrière un wrapper.

2. Problèmes initiaux dans A-CorePilot

- Des includes PROG/I affichaient zéro caller alors que leur programme principal les incluait.
- Un zéro pouvait aussi représenter un appel SAP échoué ou une réponse inutilisable.
- Des composants techniques de classes étaient comptés comme callers de leur propre classe.
- Les relations incoming/outgoing ne correspondaient pas toujours à SAP GUI.
- Le graphe pouvait sembler vide après navigation malgré sa persistance.
- ATC sur près de cent objets rendait le scan long et vulnérable aux interruptions.

Affichage | Signification adoptée |

0 | La recherche SAP a réussi et aucune relation vérifiée n'a été trouvée. |

- | Valeur inconnue : type non supporté, appel échoué, réponse inutilisable ou scan non terminé. |

n > 0 | Nombre d'objets utilisateurs uniques associés à des relations SAP vérifiées. |

Principe : l'absence de preuve n'est pas une preuve d'absence. Un échec ne doit jamais être converti en zéro.

3. Provider ABAP de dépendances

Le where-used ADT générique n'étant pas cohérent pour tous les types techniques, une interface et une classe provider ont été créées côté SAP autour de RS_EU_CROSSREF.

A-CorePilot -> /zacore/dependencies -> ZCL_ACORE_DEPENDENCY_HTTP -> ZCL_ACORE_DEPENDENCY_SAP

-> RS_EU_CROSSREF

L'interface ZIF_ACORE_DEPENDENCY_PROVIDER expose GET_USED_BY avec le nom et la classe de l'objet en entrée, puis les relations, le statut, le message et le compteur en sortie.

CALL FUNCTION 'RS_EU_CROSSREF'

EXPORTING

i_find_obj_cls = lv_object_class

no_dialog = 'X'

TABLES

i_findstrings = lt_findstrings

o_founds = lt_founds.

Les résultats RSFINDLST sont normalisés, les préfixes comme \IC: sont retirés, puis les relations sont dédoublées par source, cible, classes et nature. Le compteur est le nombre de sources uniques et non le nombre brut d'occurrences.

Objet testé | Classe | Résultat observé |

ZCAP_1_TOP, ZCAP_1_SS, ZCAP_1_CLA | REPS | Un report parent vérifié pour chaque include. |

ZCL_ZAJ_BIAT_RESERVATI_DPC | CLAS | Trois utilisateurs uniques pendant le test. |

ZCL_ACCESS_PAYMENTS | CLAS | Composants générés internes éliminés du compteur métier. |

4. Graphe de dépendances fiable

Catégorie | Sources | Effet |

SAP Verified | SAP_CROSSREF | Alimente Where Used, Dependencies, Total Impact, Depth, cycles et chemins. |

Candidate | Regex, ATC, ADT générique, métadonnées ou LLM | Visible sur demande, mais n'augmente aucun compteur de production. |

- Where Used / incoming : objets qui utilisent l'objet sélectionné.

- Dependencies / outgoing : objets utilisés par l'objet sélectionné.

- Total Impact : ensemble transitif des utilisateurs vérifiés susceptibles d'être affectés.

L'option Candidate edges est désactivée par défaut. Le cache et le graphe sont versionnés, écrits atomiquement et sauvegardés par checkpoints partial. Un seul scan peut fonctionner à la fois et les erreurs inattendues sont remontées dans l'interface.

5. Classification R5 sans LLM

La classification LLM a été remplacée par des règles déterministes basées sur les métadonnées SAP, les findings ATC, le code ABAP et les conventions de nommage. Chaque décision conserve son type, sa confiance, sa source et sa preuve.

Type | Exemples de preuve | Risque |

CLASSICAL_MODIFICATION | Finding ATC de modification directe | 1,00 |

IMPLICIT_ENHANCEMENT | Métadonnée/finding « implicit enhancement » | 0,85 |

EXPLICIT_ENHANCEMENT | ENHANCEMENT-POINT ou ENHANCEMENT-SECTION | 0,70 |

BADI_IMPLEMENTATION | IF_EX_*, GET BADI, CALL BADI | 0,55 |

CUSTOM_DEVELOPMENT | Objet Z/Y sans preuve plus forte | 0,30 |

La colonne R5 Type est disponible dans Object Inventory. Son infobulle affiche la provenance et la preuve.

6. Exercice Clean Core sur le report ALV

6.1 État initial et baseline ATC

ZPROG_ALV_YME utilisait un ALV classique, des structures SLIS_*, des textes codés en dur, des SELECT SINGLE et des accès directs à BKPF, BSEG, EKPO, T030K, A003 et KONP. ATC signalait des APIs internes/classiques, lectures DDIC, sélections potentiellement non uniques et éléments de texte manquants.

6.2 Protection du programme original

Une copie cohérente a été créée avec Source, Text Elements et Includes :

```
ZPROG_ALV_YME_CC  
ZPROG_ALV_YME_CC_TOP  
ZPROG_ALV_YME_CC_SS  
ZPROG_ALV_YME_CC_CLA
```

L'original est resté la référence fonctionnelle. La gestion des versions SAP a été utilisée pour comparer la version active et les versions locales.

6.3 Structure et corrections syntaxiques

```
REPORT zprog_alv_yme_cc.  
INCLUDE zprog_alv_yme_cc_top.  
INCLUDE zprog_alv_yme_cc_ss.  
INCLUDE zprog_alv_yme_cc_cla.
```

```
INITIALIZATION.  
CREATE OBJECT go_report.  
START-OF-SELECTION.  
go_report->get_data( ).  
go_report->show_alv( ).
```

- Ajout de CLASS lcl_report DEFINITION DEFERRED. avant la référence à la classe.
- Correction des blocs DATA, virgules et points.
- Déclaration publique des méthodes appelées.
- Suppression de l'événement inexistant PF_STATUS_SET sur CL_GUI_ALV_GRID.
- Activation des includes avant le programme principal.

Résultat : « No syntax errors found in Report ZPROG_ALV_YME_CC », puis affichage de l'écran de sélection avec F8.

6.4 Text Pool

Les chaînes visibles ont été externalisées dans les Text Symbols 001 à 024 : titres des blocs, messages « Aucune donnée trouvée » et « Erreur lors de l'affichage ALV », ainsi que les titres des colonnes comptables et fiscales.

MESSAGE text-003 TYPE 'S' DISPLAY LIKE 'E'.

Le Text Pool a été enregistré et activé en français avant de relancer ATC.

6.5 Modernisation de l'ALV

Les appels REUSE_ALV_* et les structures SLIS_* ont été retirés au profit de SALV :

```
DATA lo_alv TYPE REF TO cl_salv_table.  
cl_salv_table=>factory(  
  IMPORTING r_salv_table = lo_alv  
  CHANGING t_table      = mt_result ).  
lo_alv->get_columns( )->set_optimize( abap_true ).  
lo_alv->get_functions( )->set_all( abap_true ).  
lo_alv->display( ).
```

SALV modernise l'affichage SAP GUI. Une cible ABAP Cloud stricte peut toutefois nécessiter RAP/Fiori.

7. Recherche des APIs Released

Objet interne | Successeur étudié | Conclusion |
BKPF | I_JournalEntry | Released ; sémantique à valider. |
BSEG | I_OperationalAcctgDocItem | Released ; candidat pour les postes comptables. |
EKPO | I_PurchaseOrderItemAPI01 | Released ; candidat pour les postes de commande. |
KONP | I_SlsPrcgConditionRecord | Released, mais aucune valeur exploitable pour le test fiscal. |
T030K / A003 | Aucun successeur confirmé | Accès legacy isolé et documenté. |
Un successeur proposé dans le Cloudification Repository n'est pas automatiquement équivalent. Les champs, filtres, cardinalités et données ont été vérifiés dans le système réel.

8. Wrapper fiscal

ZPROG_ALV_YME_CC -> ZIF_YME_TAX_PROVIDER -> ZCL_YME_TAX_LEGACY -> T030K / A003 / KONP

8.1 Interface

```
INTERFACE zif_yme_tax_provider PUBLIC.  
  METHODS get_tax_data  
  IMPORTING iv_tax_code      TYPE mwszkz  
  EXPORTING ev_tax_account   TYPE saknr  
  ev_tax_rate               TYPE kbetr  
  ev_account_found          TYPE abap_bool
```

```
ev_rate_found TYPE abap_bool.  
ENDINTERFACE.
```

8.2 Implémentation legacy

```
METHOD zif_yme_tax_provider~get_tax_data.  
CLEAR: ev_tax_account, ev_tax_rate,  
ev_account_found, ev_rate_found.
```

```
SELECT SINGLE konts FROM t030k  
WHERE ktopl = 'MXCA' AND ktosl = 'VST'  
AND land1 = 'TN' AND mwszkz = @iv_tax_code  
INTO @ev_tax_account.  
IF sy-subrc = 0. ev_account_found = abap_true. ENDIF.
```

```
SELECT SINGLE knumh FROM a003  
WHERE kappl = 'TX' AND kschl = 'MWVS'  
AND aland = 'TN' AND mwszkz = @iv_tax_code  
INTO @DATA(lv_knumh).  
IF sy-subrc <> 0. RETURN. ENDIF.
```

```
SELECT SINGLE kbetr FROM konp  
WHERE knumh = @lv_knumh AND kopos = '01'  
INTO @DATA(lv_rate).  
IF sy-subrc = 0.  
ev_tax_rate = lv_rate / 10.  
ev_rate_found = abap_true.  
ENDIF.  
ENDMETHOD.
```

8.3 Wrapper comparé à une classe simple

Critère | Classe couplée directement | Wrapper avec interface |
Dépendance du report | Tables SAP internes | Contrat ZIF_YME_TAX_PROVIDER |
Dette technique | Dispersée | Confinée dans ZCL_YME_TAX_LEGACY |
Test | Difficile à isoler | Provider testable séparément |
Remplacement futur | Modification du report | Nouvelle implémentation de l'interface |
Clean Core strict | Non | Pas encore, mais dette localisée |
Limite : un wrapper ne rend pas une table interne Released. Il confine la dette jusqu'à la disponibilité d'une API officielle compatible.

8.4 Test fonctionnel

GET_TAX_DATA a été exécutée dans SE24 avec IV_TAX_CODE = D1 :

```
EV_TAX_ACCOUNT = 4366100000  
EV_TAX_RATE = 18,00  
EV_ACCOUNT_FOUND = X
```

EV_RATE_FOUND = X

Le résultat a été comparé aux données de T030K, A003 et KONP afin de préserver le comportement métier.

9. Méthode de validation

- Contrôle syntaxique et activation de tous les includes.
- Exécution de l'original et de la copie avec les mêmes critères.
- Comparaison du nombre de lignes, des clés, du compte TVA et du taux.
- Relance ATC après chaque groupe de changements.
- Test des CDS Released dans le système sans supposer leur équivalence.
- Vérification d'un échantillon Where Used dans SAP GUI.
- Dans A-CorePilot, validation avec Candidate edges désactivé.
- Conservation des captures avant/après et justification des exceptions.

10. Tests du projet

```
npm run compile  
npm run test:graph
```

La dernière validation réalisée pendant le développement comptait 16 tests réussis et aucun échec. Les scénarios couvrent les includes, les callers anonymes rejetés, les candidats exclus des métriques, les BAdI, ATC et les catégories R5.

11. Bilan avant / après

Avant | Après |
Zéro ambigu en cas d'échec | Zéro réservé au succès vide ; tiret pour inconnu |
Sources mélangées | Métriques uniquement SAP_CROSSREF |
Regex considérée comme vérité | Regex conservée comme candidate |
Ancien cache potentiellement affiché | Cache versionné, atomique et checkpoints |
R5 basé sur LLM | R5 déterministe avec preuve |
Original modifié directement | Copie ZPROG_ALV_YME_CC isolée |
ALV classique et SLIS | SALV pour la modernisation SAP GUI |
Textes codés en dur | Text Symbols français |
Accès fiscaux dispersés | Confinés dans ZCL_YME_TAX_LEGACY |

12. Limites et prochaines étapes

- La précision dépend des index cross-reference du système SAP.
- Les dépendances dynamiques ne sont pas toutes détectables statiquement.
- Le scan est limité à 100 objets pour maîtriser la durée.
- Les tables du wrapper doivent être remplacées lorsqu'une API Released compatible apparaît.
- Les objets de production doivent quitter \$TMP et rejoindre un package transportable.
- Une cible stricte doit étudier RAP/Fiori et les contraintes ABAP Cloud.

13. Architecture technique détaillée d'A-CorePilot

A-CorePilot est une extension VS Code organisée en couches. Cette séparation évite que l'interface utilisateur connaisse les détails des protocoles SAP et permet de tester les règles métier indépendamment du webview.

Couche | Responsabilité | Principaux composants |
Présentation | Inventaire, progression du scan, graphes, filtres et détails. |
ObjectInventory, DependencyGraphView, SidebarProvider |
Orchestration | Enchaînement des phases, gestion des erreurs et sauvegardes. | ScannerAgent, DashboardPanel |
Analyse | ATC, dette, classification R5 et enrichissement. | AtcAgent, DebtAnalystAgent, ModificationTypeClassifier |
Découverte SAP | Recherche Z/Y, source, métadonnées et relations. | ObjectDiscovery, AdtClient, AtcRunner |
Modèle graphe | Noeuds, arêtes, métriques, cycles et impact transitif. | DependencyGraph, GraphSerializer |
Persistance | Cache versionné des objets et du graphe. | scan-results.json, dependency-graph.json |
SAP backend | Where-used normalisé et exposé en JSON. | ZIF_ACORE_DEPENDENCY_PROVIDER, ZCL_ACORE_DEPENDENCY_SAP, ZCL_ACORE_DEPENDENCY_HTTP |

13.1 Flux général d'un scan

- Lecture de l'hôte, du client et du nom utilisateur depuis les paramètres VS Code.
- Lecture du mot de passe depuis le stockage sécurisé.
- Ping de l'endpoint ADT afin de distinguer un problème de connexion d'une erreur d'analyse.
- Invalidation logique de l'ancien cache par création d'un checkpoint Discovery vide.
- Recherche des objets Z* et Y* des types PROG, CLAS, FUGR et INTF.
- Déduplication par type et nom.
- Limitation de l'échantillon à 100 objets pendant la phase de test.
- Enrichissement par petits lots pour protéger le système SAP.
- Appel du provider SAP pour les callers vérifiés.
- Lecture du source et calcul LOC lorsque le type le permet.
- Extraction des relations candidates depuis le source.
- Lecture de l'historique de modification lorsque l'objet possède un endpoint autonome.
- Construction et sauvegarde d'un premier graphe.
- Exécution ATC uniquement sur les objets autonomes.
- Enrichissement CRV lorsqu'un successeur est disponible.
- Calcul de la dette et de l'effort.
- Reconstruction du graphe avec les données finales.
- Écriture atomique du résultat complet et notification de l'interface.

13.2 Modèle d'un objet enrichi

EnrichedObject
name, type, uri, packageName, description
callerCount, callerScanStatus, callers
calleeCount, dependencies, impactCount, graphDepth

loc, lastChangedDate, changesLast12Months
atcFindings, classification
modType, modTypeConfidence, modTypeSource, modTypeEvidence
debtScore, effortSP, riskScore, riskLevel
crvEntry

La présence simultanée de la valeur et de son statut est essentielle. Sans callerScanStatus, la valeur numérique seule ne permet pas de savoir si zéro signifie « aucun caller » ou « analyse impossible ».

14. Chronologie complète des travaux

- Lancement du projet depuis le workspace et observation de l'inventaire.
- Identification d'objets affichés avec zéro caller ou un tiret.
- Analyse spécifique des sous-types PROG/I.
- Vérification manuelle des reports ZCAP_1 et de ses includes.
- Constat que SAP GUI ne propose pas toujours une Where-Used List directe sur un include.
- Comparaison des relations du graphe avec SE24 et les résultats SAP GUI.
- Identification du faux comptage des composants générés de classes.
- Test de RS_EU_CROSSREF dans un programme ABAP dédié.
- Étude des structures RINFOBJ, RSFIND et RSFINDLST.
- Création d'une structure de relation A-CorePilot et de son type table.
- Création de l'interface du provider de dépendances.
- Création de l'implémentation SAP.
- Normalisation et déduplication des utilisateurs.
- Création du handler HTTP et publication sous /zacore/dependencies.
- Activation du service ICF et test depuis le navigateur/terminal.
- Connexion de l'extension TypeScript au nouvel endpoint.
- Test de ZCAP_1_TOP, puis des classes et DPC.
- Correction de la distinction entre relation technique interne et caller externe.
- Séparation des arêtes vérifiées et candidates.
- Recalcul de Where Used, Dependencies et Impact depuis les seules preuves SAP.
- Ajout du cache versionné et des checkpoints.
- Blocage des scans simultanés.
- Création du classifieur déterministe R5.
- Ajout de la colonne R5 Type et de son explication.
- Compilation et exécution des tests automatisés.
- Démarrage de l'exercice ALV par une baseline ATC.
- Création d'une copie séparée du report.
- Correction progressive des includes et de la classe locale.
- Externalisation des textes.
- Modernisation de l'ALV.
- Recherche des successeurs Released.
- Test réel des vues CDS proposées.
- Constat d'absence d'équivalence fiscale suffisante.
- Création du wrapper fiscal.
- Test du code TVA D1 et comparaison des résultats.
- Nouvelle exécution ATC et documentation des findings résiduels.

15. Algorithme détaillé du Where Used vérifié

15.1 Entrées

- Nom normalisé de l'objet, par exemple ZCAP_1_TOP.
- Classe Repository attendue, par exemple REPS ou CLAS.

15.2 États de sortie

Statut | Interprétation | Affichage |

SUCCESS avec relations | Recherche réussie et callers identifiés. | Nombre positif |

SUCCESS sans relation | Recherche réussie, aucun caller. | 0 |

UNSUPPORTED | Le mécanisme ne peut pas analyser ce type. | - |

NOT_FOUND | Objet absent ou mauvais couple nom/classe. | - |

FAILED | Erreur technique ou réponse inutilisable. | - |

15.3 Pseudo-code

```
validate object name
choose repository class
call RS_EU_CROSSREF
map SY-SUBRC to explicit status
for every RSFINDLST row:
    normalize source and target
    remove repository prefixes
    reject empty or self-generated artifacts
    create typed relation
    sort relations by semantic identity
    delete adjacent duplicates
    count unique source objects
return status, count, relations and evidence
```

15.4 Exemple JSON

```
{
  "objectName": "ZCAP_1_TOP",
  "objectClass": "REPS",
  "status": "SUCCESS",
  "message": "1 verified user(s), 1 relation(s)",
  "usedByCount": 1,
  "relations": [{
    "sourceName": "ZCAP_1",
    "sourceClass": "P",
    "targetName": "ZCAP_1_TOP",
    "targetClass": "I",
    "relationKind": "INCLUDES",
    "sourceLine": 22,
```

```
"verified": true
}}
}
```

16. Construction et métriques du graphe

16.1 Identité canonique

Un noeud est identifié par TYPE::NOM. Le type est ramené à sa base : PROG/P et PROG/I deviennent des noeuds de famille PROG, tout en conservant leur nature dans l'inventaire. Les noms sont trimés et convertis en majuscules.

16.2 Déduplication des arêtes

Une arête est identifiée par source, cible et nature. Lorsque plusieurs preuves décrivent la même relation, leurs sources et leurs extraits sont fusionnés. Si une preuve SAP_CROSSREF arrive après une candidate, elle devient la provenance principale et l'arête passe à l'état vérifié.

16.3 Formules des métriques

Where Used(X) = nombre de sources uniques des arêtes vérifiées vers X
Dependencies(X) = nombre de cibles uniques des arêtes vérifiées depuis X
Impact(X) = fermeture transitive inverse depuis X
Depth(X) = plus longue profondeur vérifiée atteignable depuis X

Les cycles sont recherchés uniquement dans le sous-graphe vérifié. Cela empêche une simple occurrence regex de fabriquer un cycle de production.

16.4 Includes

Pour un report principal qui inclut TOP, SS et CLA, la direction correcte est :

```
ZCAP_1 --INCLUDES--> ZCAP_1_TOP
ZCAP_1 --INCLUDES--> ZCAP_1_SS
ZCAP_1 --INCLUDES--> ZCAP_1_CLA
```

Ainsi, chacun des includes possède un Where Used égal à un si le report parent est son seul utilisateur. Le report principal peut avoir zéro Where Used tout en possédant trois Dependencies : ces deux valeurs ne se contredisent pas.

17. ATC : exécution, performance et interprétation

ATC ne doit pas être exécuté séparément sur tous les sous-objets techniques lorsqu'ils sont déjà couverts par leur objet parent. Les sous-types I, S, M, F, K et J sont donc exclus de la liste des cibles autonomes.

- Concurrence limitée à deux objets pour ATC.

- Concurrence limitée à quatre objets pour l'enrichissement général.
- Échec d'un objet isolé sans annuler automatiquement tous les autres.
- Classification A/B/C/D seulement lorsque le résultat ATC est exploitable.
- Une erreur ATC laisse la classification inconnue au lieu d'attribuer A.

Priorité ATC | Points de dette | Classification utilisée |

1 | 10 | D lorsqu'au moins une erreur existe |

2 | 5 | C sans priorité 1 |

3 | 1 | B sans priorité 1 ou 2 |

Aucun finding après succès | 0 | A |

18. Dette technique et effort

Le score composite est normalisé sur une échelle proche de 0 à 100. Il combine les findings ATC, la taille, la profondeur du graphe, le type de modification et l'âge de l'objet.

ATC normalisé = $\min(1, \text{pointsATC} / 100)$

LOC normalisé = $\min(1, \text{LOC} / 2000)$

Depth normalisé = $\min(1, \text{graphDepth} / 10)$

Age normalisé = $\min(1, \text{ageDays} / 1825)$

Debt = $\text{round}(100 * ($
 $0.30 * \text{ATC_normalisé} +$
 $0.20 * \text{LOC_normalisé} +$
 $0.20 * \text{Depth_normalisé} +$
 $0.20 * \text{ModType_multiplier} +$
 $0.10 * \text{Age_normalisé}))$

L'effort de base est de 1, 3, 8 ou 21 story points pour A, B, C ou D. Un multiplicateur dépendant de la profondeur, borné à deux, augmente l'effort des objets fortement couplés.

Gouvernance : ces coefficients sont des règles de priorisation du projet. Ils doivent être validés par l'entreprise avant d'être utilisés pour engager un budget ou décider automatiquement d'une migration.

19. R5 : règles, priorité et explicabilité

Le classifieur examine les preuves dans un ordre contrôlé. Une modification classique détectée par ATC est prioritaire. Viennent ensuite les enhancements explicites ou implicites, les BAdI, les wrappers et enfin le développement personnalisé par défaut.

```
if ATC says modified SAP standard -> CLASSICAL_MODIFICATION / high
else if exact implicit evidence -> IMPLICIT_ENHANCEMENT / high
else if ENHANCEMENT-POINT -> EXPLICIT_ENHANCEMENT / high
else if IF_EX_* or GET/CALL BADI -> BADI_IMPLEMENTATION / high
else if ZCL_IM_* naming only -> BADI_IMPLEMENTATION / medium
else if wrapper name + delegation -> WRAPPER_PROXY / high
else if Z/Y namespace -> CUSTOM_DEVELOPMENT / medium
```

else

-> CUSTOM_DEVELOPMENT / low

Les commentaires ABAP sont retirés avant l'analyse afin qu'un exemple commenté ne déclenche pas une classification. Une syntaxe ENHANCEMENT ambiguë ne permet pas d'inventer si le hook est implicite ou explicite : elle reçoit une confiance faible.

20. Exercice ALV : procédure détaillée dans SAP GUI

20.1 Baseline

- Ouvrir le report original dans SE38 ou SE80.
- Effectuer Ctrl+F2 pour le contrôle syntaxique.
- Exécuter F8 avec un jeu de critères reproductible.
- Capturer le nombre de lignes et quelques résultats métier.
- Exécuter ATC et enregistrer les findings initiaux.
- Ne pas corriger directement l'original.

20.2 Création de la copie

- Copier vers ZPROG_ALV_YME_CC.
- Cocher Source, Text Elements et Includes.
- Créer les includes avec une nomenclature homogène.
- Vérifier que le report principal référence uniquement les nouveaux includes.
- Attribuer un package de test ou une demande de transport selon les règles du système.

20.3 Ordre d'activation

- TOP : types et déclaration différée.
- SS : écran de sélection.
- CLA : classe locale.
- Text Pool du programme principal.
- Report principal.
- Ctrl+F2 global puis Ctrl+F3.
- F8 et comparaison avec la baseline.

21. Organisation recommandée du code ALV

21.1 TOP

```
CLASS lcl_report DEFINITION DEFERRED.
```

```
TYPES: BEGIN OF lty_result,  
company_code      TYPE buks,  
document_number   TYPE belnr_d,  
fiscal_year       TYPE gjahr,  
posting_date      TYPE budat,  
document_date     TYPE bldat,  
tax_code          TYPE mwskz,
```

```
tax_account      TYPE saknr,  
tax_rate         TYPE kbetr,  
END OF lty_result.
```

```
TYPES lty_result_tab TYPE STANDARD TABLE OF lty_result  
WITH EMPTY KEY.
```

```
DATA go_report TYPE REF TO lcl_report.
```

21.2 Sélection

```
SELECTION-SCREEN BEGIN OF BLOCK b1  
WITH FRAME TITLE text-001.  
SELECT-OPTIONS:  
s_ste  FOR gv_company_code,  
s_npc  FOR gv_document_number,  
s_excpt FOR gv_fiscal_year DEFAULT '2025'.  
PARAMETERS p_ldg TYPE rldnr.  
SELECTION-SCREEN END OF BLOCK b1.
```

La convention retenue utilise S_ pour SELECT-OPTIONS et P_ pour PARAMETERS. Les variables utilisées par FOR doivent être déclarées avant l'include SS.

21.3 Classe locale

```
CLASS lcl_report DEFINITION FINAL.  
PUBLIC SECTION.  
METHODS constructor  
IMPORTING io_tax_provider TYPE REF TO zif_yme_tax_provider.  
METHODS run.  
PRIVATE SECTION.  
DATA mt_result TYPE lty_result_tab.  
DATA mo_tax_provider TYPE REF TO zif_yme_tax_provider.  
METHODS get_data.  
METHODS enrich_data.  
METHODS show_alv.  
ENDCLASS.
```

L'injection par interface évite que la classe métier construise elle-même sa dépendance legacy. Le programme principal choisit l'implémentation, tandis que le report consomme uniquement le contrat.

22. Wrapper fiscal : code d'intégration complet

22.1 Injection

```
INITIALIZATION.  
go_report = NEW lcl_report(
```

```
io_tax_provider = NEW zcl_yme_tax_legacy( ) ).
```

```
START-OF-SELECTION.
```

```
go_report->run( ).
```

22.2 Orchestration

```
METHOD run.
```

```
get_data( ).
```

```
IF mt_result IS INITIAL.
```

```
MESSAGE text-003 TYPE 'S' DISPLAY LIKE 'E'.
```

```
RETURN.
```

```
ENDIF.
```

```
enrich_data( ).
```

```
show_alv( ).
```

```
ENDMETHOD.
```

22.3 Enrichissement

```
METHOD enrich_data.
```

```
LOOP AT mt_result ASSIGNING FIELD-SYMBOL(<result>).
```

```
IF <result>-tax_code IS INITIAL.
```

```
CONTINUE.
```

```
ENDIF.
```

```
mo_tax_provider->get_tax_data(
```

```
EXPORTING
```

```
iv_tax_code      = <result>-tax_code
```

```
IMPORTING
```

```
ev_tax_account   = <result>-tax_account
```

```
ev_tax_rate      = <result>-tax_rate
```

```
ev_account_found = DATA(lv_account_found)
```

```
ev_rate_found    = DATA(lv_rate_found) ).
```

```
ENDLOOP.
```

```
ENDMETHOD.
```

22.4 Remplacement futur

Lorsque SAP fournit une API Released fonctionnellement équivalente, une classe ZCL_YME_TAX_RELEASED peut implémenter la même interface. Le report ne change pas ; seule l'instanciation est remplacée.

23. Analyse des SELECT SINGLE

Le finding « SELECT SINGLE is possibly not unique » signifie que le prédicat ne couvre pas une clé unique. Il ne faut pas le masquer sans règle métier.

Situation | Correction |

La clé complète est connue | Ajouter tous les champs de clé au WHERE. |

Plusieurs lignes sont métierment possibles | Lire la liste et traiter explicitement la cardinalité. |
Une seule ligne doit être choisie | SELECT UP TO 1 ROWS avec ORDER BY déterministe. |
Lecture répétée dans LOOP | Précharger par lots ou joindre via une source Released adaptée. |
Dans le wrapper fiscal, la sélection doit également considérer les dates de validité si elles appartiennent réellement à la source utilisée. L'erreur rencontrée sur DATAB a montré qu'il ne faut jamais supposer qu'un champ existe dans A003 sans vérifier le dictionnaire du système.

24. Matrice de tests

ID | Scénario | Résultat attendu |
DG-01 | Include avec un report parent | Where Used = 1, relation INCLUDES vérifiée |
DG-02 | Objet sans caller après SUCCESS | Where Used = 0 |
DG-03 | Provider indisponible | Tired ou statut failed, jamais zéro confirmé |
DG-04 | Relation regex sans SAP_CROSSREF | Candidate, aucun impact métrique |
DG-05 | Composant généré de classe | Pas de faux caller de la classe racine |
DG-06 | Deux preuves pour la même arête | Une seule arête avec preuves fusionnées |
R5-01 | Finding modification standard | CLASSICAL_MODIFICATION / high |
R5-02 | INTERFACES IF_EX_* | BADI_IMPLEMENTATION / high |
R5-03 | Nom wrapper avec délégation | WRAPPER_PROXY / high |
R5-04 | Objet Z standard sans signal | CUSTOM_DEVELOPMENT / medium |
ALV-01 | Ctrl+F2 sur la copie | Aucune erreur syntaxique |
ALV-02 | F8 avec baseline | Même résultat fonctionnel |
TAX-01 | Code TVA D1 | Compte 4366100000, taux 18, flags X |
ATC-01 | Text Pool activé | Disparition des findings texte concernés |

25. Diagnostic des incidents

Symptôme | Cause probable | Action |
Le graphe est vide | Cache ancien/incompatible ou scan partiel | Recharger l'extension et relancer un scan complet |
Le scan semble disparaître en changeant d'onglet | État UI non restauré | Vérifier la barre globale et les messages scanProgress |
Zéro inhabituel | Succès réellement vide ou mauvaise classe Repository | Contrôler status, message, objectClass et relations JSON |
Beaucoup de dépendances fausses | Candidate edges activé | Désactiver l'option et examiner SAP Verified |
ATC très long | Trop de cibles ou variante lourde | Exclure les sous-objets, réduire l'échantillon, utiliser la variante validée |
Ancienne interface après compilation | Extension Host non rechargé | Developer: Reload Window ou relancer F5 |
No values found sur un CDS | Successeur non équivalent ou filtres incorrects | Comparer modèle, données et sémantique ; ne pas forcer le remplacement |

26. Checklist de mise en production

26.1 Sécurité

- Protéger l'endpoint /zcore/dependencies par authentification et autorisations adaptées.
- Ne jamais journaliser les mots de passe ou secrets SAP.
- Utiliser HTTPS avec un certificat reconnu.
- Limiter l'accès Repository selon les rôles de l'utilisateur technique.
- Tracer les erreurs sans exposer le source ABAP sensible.

26.2 Performance

- Valider les limites de concurrence avec l'équipe Basis.
- Mesurer le temps moyen et le taux d'échec sur un package pilote.
- Prévoir reprise, timeout, backoff et cache selon la charge.
- Éviter ATC complet synchrone pour des milliers d'objets.
- Prévoir scan incrémental par date ou transport.

26.3 Qualité des données

- Afficher date, système, client et version du scan.
- Ne mélanger ni environnements ni clients SAP dans le même cache.
- Conserver la provenance de chaque arête.
- Auditer un échantillon contre SAP GUI.
- Définir un seuil de couverture avant usage décisionnel.

26.4 Transport et gouvernance

- Déplacer les objets depuis \$TMP vers un package transportable.
- Créer une demande dédiée et documentée.
- Séparer objets backend SAP, extension VS Code et configuration.
- Faire valider le wrapper legacy par l'architecture Clean Core.
- Créer une date de révision pour chaque exception non Released.

27. Livrables réalisés

- extension A-CorePilot compilable ;
- inventaire des objets et statuts where-used ;
- provider SAP de dépendances et service HTTP ;
- graphe vérifié avec candidats séparés ;
- cache résilient et scans protégés ;
- classifieur R5 déterministe ;
- tests TypeScript du graphe et de R5 ;
- copie ZPROG_ALV_YME_CC et ses includes ;
- Text Pool français ;
- modernisation SALV ;
- analyse Cloudification Repository ;
- interface et classe wrapper fiscal ;
- preuve fonctionnelle D1 ;
- comparaison avant/après et limites documentées.

28. Glossaire

Terme | Définition |

ADT | ABAP Development Tools et services HTTP utilisés par les outils de développement. |

ATC | ABAP Test Cockpit, moteur de contrôles syntaxiques, qualité et cloudification. |

Caller / Used By | Objet qui utilise l'objet analysé ; relation entrante. |

Dependency | Objet utilisé par l'objet analysé ; relation sortante. |

Clean Core | Approche visant à réduire les modifications du standard et à consommer des contrats stables. |

Released API | Objet explicitement publié par SAP pour un scénario et un contrat d'usage. |

Wrapper | Classe qui cache un composant legacy derrière une interface stable. |

Candidate | Relation plausible mais non confirmée par le provider Repository. |

SAP Verified | Relation confirmée par la source SAP_CROSSREF du projet. |

Impact transitif | Ensemble des utilisateurs directs et indirects potentiellement affectés.

|

CRV | Cloudification Repository Viewer, utilisé pour rechercher statut et successeur. |

R5 | Dimension de risque liée au type de modification ou d'extension. |

29. Conclusion

A-CorePilot est passé d'un prototype aux compteurs difficiles à interpréter vers une chaîne où la provenance des relations est visible et où seules les preuves SAP vérifiées influencent les métriques. L'exercice ALV a appliqué une migration progressive : préserver l'original, stabiliser une copie, corriger les problèmes, rechercher et tester les APIs Released, puis isoler la dette non remplaçable.

Le résultat n'est pas présenté comme une conformité Clean Core absolue. Il constitue une base auditable, testable et évolutive dont les limites sont documentées.

Rapport généré à partir des travaux réalisés dans le workspace A-CorePilot et des validations effectuées dans SAP Logon.